

Unit 4

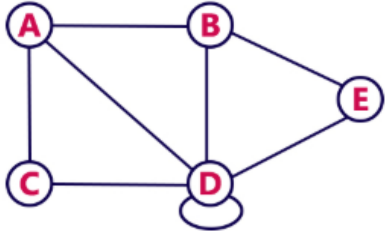
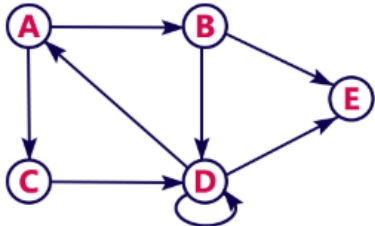
NON LINEAR DATA STRUCTURES – GRAPHS

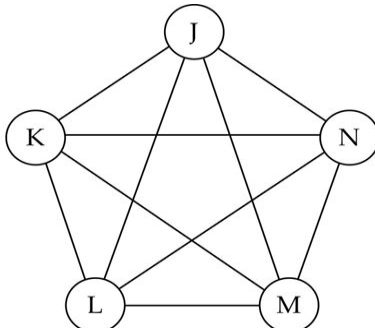
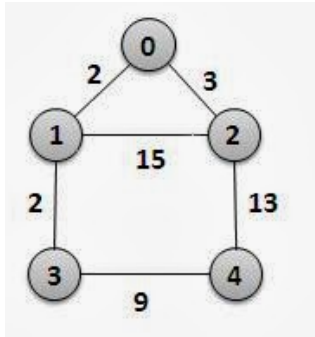
Course objective: To apply Graph data structure

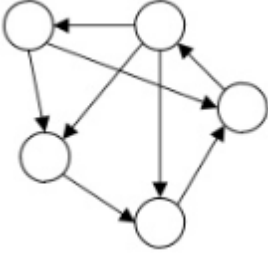
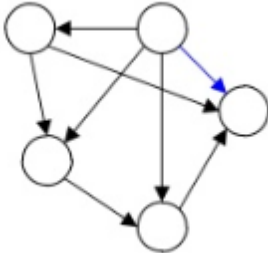
Course Outcome: Apply the different non-linear data structures to problem solutions.

Bloom's Taxonomy Level: K1, K2, K3, K4

PART A

Q.N	Question	BTL									
1.	What is a graph? A Graph is a non-linear data structure consisting of nodes and edges. The nodes are sometimes also referred to as vertices and the edges are lines or arcs that connect any two nodes in the graph.	K1									
2.	List the types of graph. <u>Types:</u> Directed graph Undirected graph Complete graph Cyclic graph Acyclic graph	K1									
3.	Consider the graph given below. Create the adjacency matrix of it  $ \begin{matrix} & \begin{matrix} A & B & C & D & E \end{matrix} \\ \begin{matrix} A \\ B \\ C \\ D \\ E \end{matrix} & \begin{pmatrix} 0 & 1 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 1 & 0 \\ 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & 0 & 1 & 0 \end{pmatrix} \end{matrix} $	K6									
4.	Find out the in-degree and out-degree of each node in the given graph  <table border="1" style="width: 100%; text-align: center;"> <thead> <tr> <th>Vertex</th><th>Indegree</th><th>Outdegree</th></tr> </thead> <tbody> <tr> <td>A</td><td>1</td><td>2</td></tr> <tr> <td>B</td><td>1</td><td>2</td></tr> </tbody> </table>	Vertex	Indegree	Outdegree	A	1	2	B	1	2	K1
Vertex	Indegree	Outdegree									
A	1	2									
B	1	2									

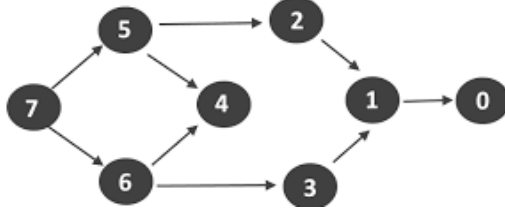
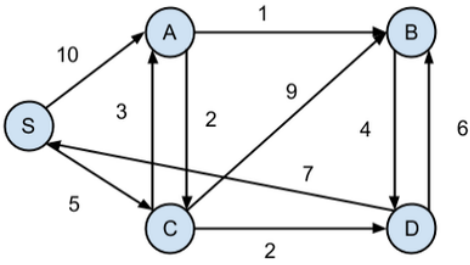
	<table><tr><td>C</td><td>1</td><td>1</td></tr><tr><td>D</td><td>3</td><td>3</td></tr><tr><td>E</td><td>2</td><td>0</td></tr></table>	C	1	1	D	3	3	E	2	0																												
C	1	1																																				
D	3	3																																				
E	2	0																																				
5.	Create a complete undirected graph having five nodes A complete graph has an edge between every pair of vertices. For a given number of vertices, there's a unique complete graph, which is often written as K_n, where n is the number of vertices. 	K6																																				
6.	Given the following adjacency matrix, draw the weighted graph. <table><tr><td></td><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td></tr><tr><td>0</td><td>0</td><td>2</td><td>3</td><td>0</td><td>0</td></tr><tr><td>1</td><td>2</td><td>0</td><td>15</td><td>2</td><td>0</td></tr><tr><td>2</td><td>3</td><td>15</td><td>0</td><td>0</td><td>13</td></tr><tr><td>3</td><td>0</td><td>2</td><td>0</td><td>0</td><td>9</td></tr><tr><td>4</td><td>0</td><td>0</td><td>13</td><td>9</td><td>0</td></tr></table> 		0	1	2	3	4	0	0	2	3	0	0	1	2	0	15	2	0	2	3	15	0	0	13	3	0	2	0	0	9	4	0	0	13	9	0	K3
	0	1	2	3	4																																	
0	0	2	3	0	0																																	
1	2	0	15	2	0																																	
2	3	15	0	0	13																																	
3	0	2	0	0	9																																	
4	0	0	13	9	0																																	
7.	Give the purpose of Dijkstra's algorithm Dijkstra's algorithm is called the single-source shortest path. It is also known as the single source shortest path problem. It computes length of the shortest path from the source to each of the remaining vertices in the graph	K2																																				
8.	Define cyclic and acyclic graph A cyclic graph is a directed graph which contains a cycle (a path from at least one node back to itself). An acyclic graph is a directed graph which contains absolutely no cycle, that is no node can be traversed back to itself.	K1																																				
9.	Classify strongly connected and weakly connected graph	K4																																				

	strongly connected graph	weakly connected graph	
	A directed graph is <u>strongly connected</u> if there is a path between all pairs of vertices.(there is a path from A to B and B to A whenever A and B are vertices in the graph)	A directed graph is <u>weakly connected</u> If there ia a path from every two vertices of the graph when the direction of edges are disregarded	
			
10.	Define the length of the graph. length of the graph is defined as the Number of edges in the longest path from the root node to the leaf node		K1
11.	Define minimum spanning tree. A spanning tree is a subset of Graph G, which has all the vertices covered with minimum possible number of edges. Hence, a spanning tree does not have cycles In a weighted graph, a <u>minimum spanning tree</u> is a spanning tree that has minimum weight than all other spanning trees of the same graph.		K1
12.	State the principle of Topological sorting. A topological sort is a linear ordering of vertices in a directed acyclic graph such that if there is a path from v_i to v_j then v_j appears after v_i in the linear ordering.		K1
13.	Explain procedure for Depth first search algorithm. Depth first search works by selecting one vertex V of G as a start vertex; V is marked veisited . Then each unvisited vertex adjacent to V is searched in turn using depth first search recursively. This process continues until no unvisited vertices are encountered.		K2
14.	What are the two ways of representing a graph. 1. Adjacency matrix 2. Adjacency list		K1
15.	Define indegree and outdegree of a graph. • <u>The indegree</u> of a vertex V , is the number of edges entering in to the vertex V • <u>The outdegree</u> of the vertex v, is the number of edges exiting from the vertex V.		K1
16.	List out the applications of Graph. • To define the flow of computation. • To represent networks of communication. • To represent data organization. • Widely using in Google maps, Facebook, WWW		K1
17.	Define Prim's algorithm.		K1

	Prim's Algorithm is a greedy algorithm that is used to find the minimum spanning tree from a graph. Prim's algorithm finds the subset of edges that includes every vertex of the graph such that the sum of the weights of the edges can be minimized.																					
18.	Mention the procedure to find minimum spanning tree using Kruskal's Algorithm. - First, sort all the edges from low weight to high. - Now, take the edge with the lowest weight and add it to the spanning tree. If the edge to be added creates a cycle, then reject the edge. - Continue to add the edges until we reach all vertices, and a minimum spanning tree is created.	K1																				
19.	Differentiate BFS and DFS. <table><tr><th>S. No.</th><th>Parameters</th><th>Breadth First Search</th><th>Depth First Search</th></tr><tr><td>1.</td><td>Data Structure</td><td>uses Queue data structure for finding the shortest path.</td><td>uses Stack data structure.</td></tr><tr><td>2.</td><td>Definition</td><td>walk through all nodes on the same level before moving on to the next level.</td><td>traverse begins at the root node and proceeds through the nodes as far as possible until we reach the node with no unvisited nearby nodes.</td></tr><tr><td>3.</td><td>Technique</td><td>used to find a single source shortest path in an unweighted graph because, in BFS, we reach a vertex with a minimum number of edges from a source vertex.</td><td>In DFS, we might traverse through more edges to reach a destination vertex from a source.</td></tr><tr><td>4.</td><td>Approach</td><td>First In First Out.</td><td>Last In First Out.</td></tr></table>	S. No.	Parameters	Breadth First Search	Depth First Search	1.	Data Structure	uses Queue data structure for finding the shortest path.	uses Stack data structure.	2.	Definition	walk through all nodes on the same level before moving on to the next level.	traverse begins at the root node and proceeds through the nodes as far as possible until we reach the node with no unvisited nearby nodes.	3.	Technique	used to find a single source shortest path in an unweighted graph because, in BFS, we reach a vertex with a minimum number of edges from a source vertex.	In DFS, we might traverse through more edges to reach a destination vertex from a source.	4.	Approach	First In First Out.	Last In First Out.	K4
S. No.	Parameters	Breadth First Search	Depth First Search																			
1.	Data Structure	uses Queue data structure for finding the shortest path.	uses Stack data structure.																			
2.	Definition	walk through all nodes on the same level before moving on to the next level.	traverse begins at the root node and proceeds through the nodes as far as possible until we reach the node with no unvisited nearby nodes.																			
3.	Technique	used to find a single source shortest path in an unweighted graph because, in BFS, we reach a vertex with a minimum number of edges from a source vertex.	In DFS, we might traverse through more edges to reach a destination vertex from a source.																			
4.	Approach	First In First Out.	Last In First Out.																			
20.	Explain procedure for Breadth first search algorithm. Step 1: Take an Empty Queue. Step 2: Select a starting node (visiting a node) and insert it into the Queue. Step 3: Provided that the Queue is not empty, extract the node from the Queue and insert its child nodes (exploring a node) into the Queue. Step 4: Print the extracted node.	K1																				

--	--	--

PART B

Q:no	Question	BTL
1	Describe in detail about the following representations of a graph. i. Adjacency Matrix (7) ii. Adjacency list (6)	R
2	State and apply topological sorting for the given graph (16) 	
3	Differentiate depth-first search and breadth-first search traversal of a graph with suitable examples. (16)	
4	Develop an algorithm to compute the shortest path using Dijkstra's algorithm. Find single source shortest path for the given graph (16) 	
5	Show how Prim's algorithm and Kruskal's algorithm are used to find the minimum spanning tree (16)	
6	Apply appropriate algorithm to find the shortest path from 'A' to every other node of A. For the given graph (16) 